

从推理到执行：构建 AI Agent 全链路 无侵入可观测体系

Clement

阶跃星辰



极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

🕒 6月 📍 上海

👤 1000人

AiCon

全球人工智能开发与应用大会

会议时间：6月26-27日

- AI Infra 系统工程
- 多 Agent 协作与实践
- 多模态融合
- 模型训练与推理创新
- 数据平台与特征服务

🕒 8月 📍 深圳

👤 1000人

AiCon

全球人工智能开发与应用大会

会议时间：8月21-22日

- Agentic AI
- 轻量化与高效推理
- 多模态应用
- AI + IoT 场景实践
- AI 工业化落地

🕒 10月 📍 上海

👤 1200人

QCon

全球软件开发大会

会议时间：10月22-24日

- AI Agent
- Vibe Coding
- 智能可观测
- 推理基建
- 模型攻防
- AI x 创造力

🕒 12月 📍 北京

👤 1000人

AiCon

全球人工智能开发与应用大会

会议时间：12月18-19日

- 大模型架构创新
- 多模态 AI 产业融合
- 具身智能
- AI for Science
- 大模型安全

目录

01 Agent 观测新挑战

02 深度行为感知

03 全链路追踪

04 数据标准与交付

05 总结与展望

Agent 响应从 3 秒变成 120 秒，问题在哪？

Coding Agent → 沙箱执行入口 → 沙箱容器 → bash -l → ??? → 返回结果（超时 120s）

Agent 执行记录

tool_call: bash -l → 成功 ✓

只知道调了 bash，不知道为什么慢

Pod 监控

CPU 8%，内存 340MB，网络正常

看不出问题

LLM 指标

TTFT 210ms，token 正常

跟 LLM 没关系

容器日志

无输出

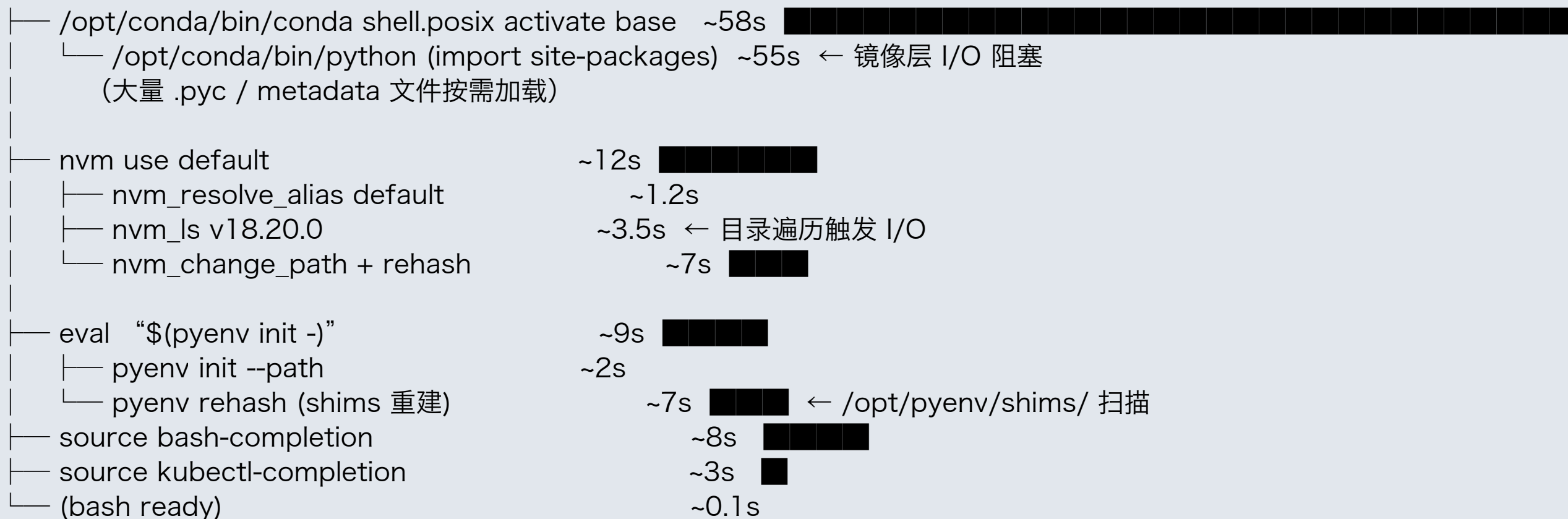
bash 初始化不写日志，一片空白

❏ 我们知道 Agent 调了 bash -l。但 bash 起来之后容器里发生了什么？

容器里实际发生了什么

最终 exec 进去逐一排查，根因其实很简单——沙箱镜像的 .bashrc 配了大量初始化脚本，每次 bash -l 要跑将近 91 秒。

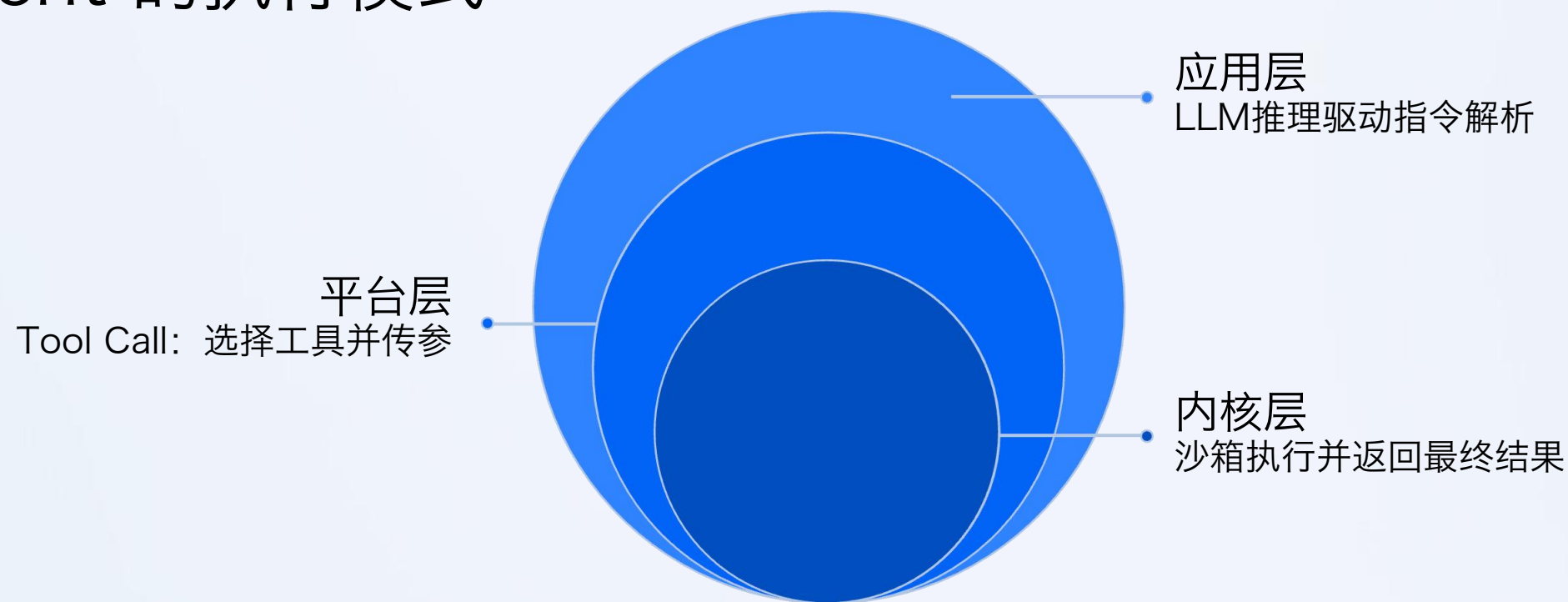
```
bash -l (PID 4521, total: ~91s)
```



01 Agent 观测新挑战

为什么 Agent 需要一种全新的可观测方案

AI Agent 的执行模式



传统微服务

确定性：相同输入 → 相同路径

静态路径：调用链在代码中静态定义

日志审计：审计调用日志即可

AI Agent

非确定性：同 Prompt → 不同 Tool Call

动态路径：LLM 实时决定调哪个工具、传什么参数

决策审计：需回溯"为什么 Agent 认为该执行这条命令"

❑ Agent 的执行链路跨越三个技术域，行为不可预测、路径动态、决策需审计—这与传统微服务的可观测性需求有本质区别。



现有可观测体系的盲区

1

盲区① 网络安全 (CNI / Firewall)

能看到: IP 到 IP 的连接

看不到: Agent 传输的 Prompt 语义、MCP 协议中的工具调用参数

2

盲区② 审计日志 (K8s Audit)

能看到: 控制平面的"意图"——谁请求了 Exec

看不到: 数据平面的"实况"——容器内部实际执行了什么命令

3

盲区③ 应用性能监控 (APM)

能看到: HTTP 延迟与错误率

看不到: Agent 思维链 (CoT) 与实际副作用 (Side Effects) 的关联

❏ Agent 的决策逻辑 (LLM) → 执行行为 (Tool) → 运行结果 (Sandbox) 的完整因果链采集需要新方案。



Agent 执行链路的四层上下文断裂

1

LLM 推理层 (OpenAI, Claude...)

2

Agent 编排层 (LangChain, AutoGen...)

断裂①: HTTPS/TLS 加密, Trace Context 缺失, Prompt 内容不可见

3

平台调度层 (kubectl exec / execd / CRI exec)

断裂②: ExecRequest 不携带 user / source / trace 身份信息

4

沙箱运行时层 (容器内进程)

❏ 核心问题——怎么把四层断裂的上下文串起来?

我们需要什么 — 从盲区倒推需求

现有盲区	需要补齐的能力
TLS 加密导致 LLM 流量黑盒	穿透 TLS — 不改代码的前提下看到 Prompt/Token
容器隔离导致运行时不可见	穿透容器 — 看到沙箱内每个进程的命令、输出、退出码
各层数据互相独立无法串联	跨层关联 — 把 LLM 对话和内核进程事件绑定到同一链路
工具调用通过 Stdio/MCP 走进程管道	Stdio 感知 — 捕获进程间管道通信（覆盖 MCP stdio 场景）
Agent 自身不知道命令是否真正成功	执行反馈 — Agent 能查询退出码/stderr/资源消耗，自主判断和排查

现实中 Agent 运行环境和类型千差万别：有 SDK 集成的 Agent、无 SDK 的自研 Agent、平台托管 Agent、Coding Agent 环境有：容器沙箱、虚拟机沙箱、MCP 场景等

 我们需要一个不依赖业务代码、不绑定特定框架、能穿透容器和 TLS 的底层采集方案，和现有的 SDK/平台方案形成互补。

现有 Agent 可观测方案全景

方案 A: SDK 侵入式

代表: LangSmith、LangFuse

优势: 语义信息最丰富: 完整 Prompt、Token、Chain-of-Thought

局限: 框架绑定、需改代码、运行时层完全无感

方案 B: 平台侧采集

代表: 百炼、Dify、Coze

优势: 零额外接入成本, 编排拓扑完整

局限: 平台锁定, 运行时层依然是黑盒

方案 C: Proxy 代理式

代表: LiteLLM Proxy、Portkey

优势: 框架无关, 接入简单

局限: 只能看 Agent 到大模型 链路, 工具调用和沙箱执行完全看不见

 缺失的能力: 运行时层 + 跨层关联 — 没有任何现有方案能回答: 容器里实际执行了什么进程? 每个进程的 stdout/stderr 是什么? 这些进程和上层 LLM 对话有什么关系?

为什么是 eBPF— 与现有方案共存

思路	做法	判断
给 SDK 加运行时采集	在 LangSmith/LangFuse 里集成进程监控	依赖框架集成，覆盖不了无 SDK 的 Agent；看不到容器内子进程
给 APM 加 Agent 语义	Datadog/Jaeger 增加 LLM trace	TLS 加密后仍是黑盒；不理解 MCP/Stdio 语义
Sidecar 容器	每 Pod 注入采集容器	资源开销随 Pod 数线性增长；需 admission webhook 改 Pod spec
eBPF ✓	DaemonSet 部署，内核级采集	零侵入 + 性能可控 + 穿透 TLS + 穿透容器

运行时层可见性

穿透容器隔离，看到沙箱内每个进程的命令行、stdout/stderr、退出码

跨层关联能力

把运行时进程事件与 LLM 对话、K8s 审计日志串联，填补上文提到的层级断裂

执行反馈闭环

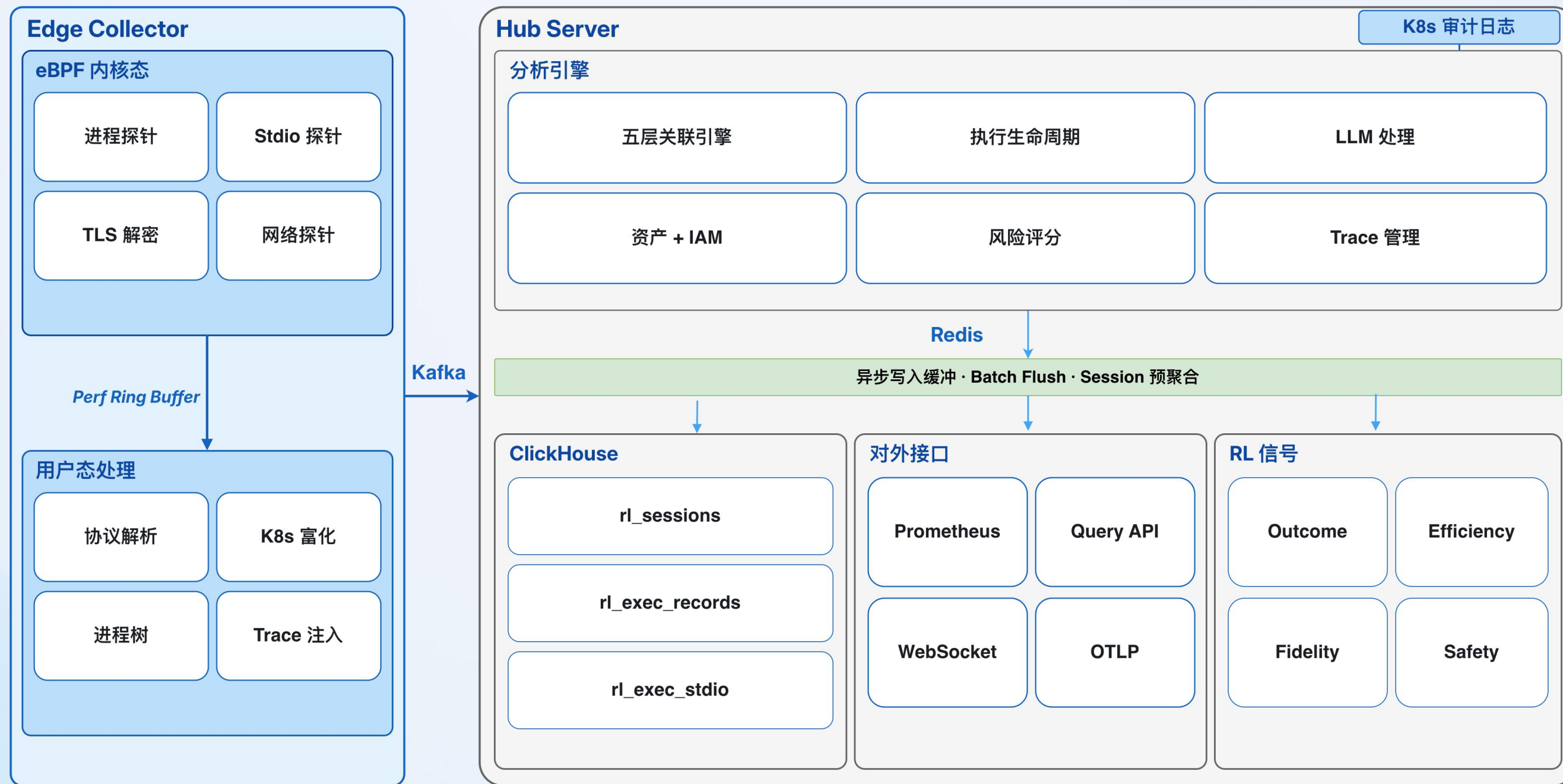
Agent 通过 API 查询退出码、stderr、资源消耗，自主判断执行效果

 最佳实践：SDK 做上层语义增强，eBPF 做底层运行时采集。两者互补，不是替代。

02 深度行为感知

如何构建 Agent 执行的“全息画像”

整体架构图

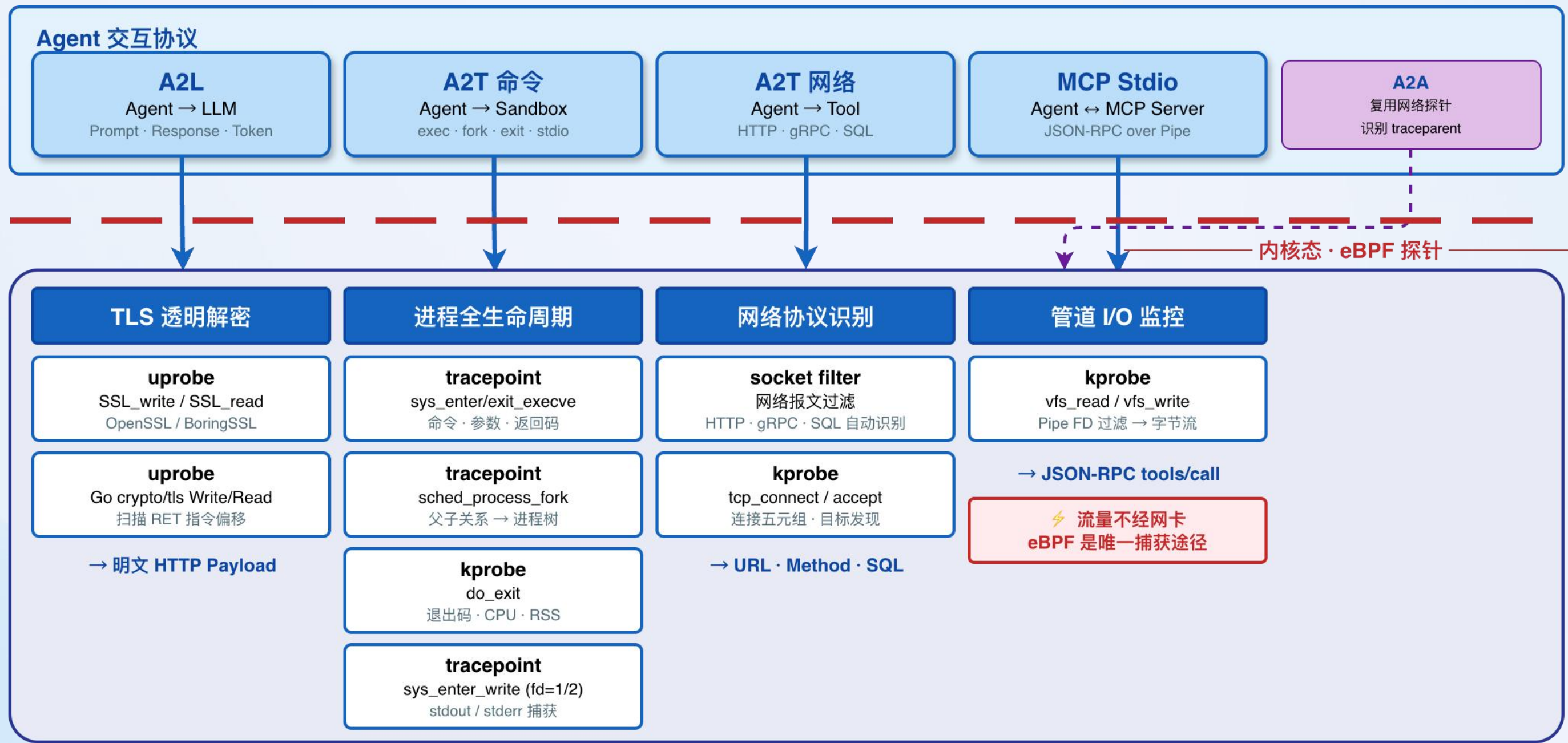


Agent 链路全景 — 四条通信链路的传感器布局

链路	场景	采集点	获取的数据
A2L	Agent → LLM API	uprobe TLS 解密 + HTTP 解析	Prompt、Token、模型名、TTFT、延迟
A2T	Agent → 沙箱执行	tracepoint (进程) + kprobe (stdio)	命令行、退出码、stdout/stderr、CPU/RSS
A2A	Agent → Agent	socket filter + Trace Context	调用关系、延迟
MCP	Agent ↔ MCP Tool (stdio pipe)	kprobe vfs_write + JSON-RPC 解析	method、工具名、参数、结果

❏ 四条链路、三种探针类型 (tracepoint / kprobe / uprobe) , 覆盖 Agent 的全部外部交互。

eBPF 采集点全景图



进程生命周期 — 三个 Tracepoint

1

sched_process_exec

PID, PPID, 命令行(comm+args), 可执行文件路径, 工作目录

2

sched_process_fork

父子 PID 关系

3

sched_process_exit

退出码, CPU 时间, RSS, 页面错误

不管 Agent 通过 `kubectl exec`、`execd` 还是 CRI 调用进入沙箱，tracepoint 都能捕获。回到开始的问题，如果有进程树，一目了然：

```
bash -l (PID 4521, exit=0, ~91s)
├── /etc/profile.d/conda.sh (PID 1235, exit=0, ~1.2s)
├── /opt/conda/bin/conda shell.posix activate base (PID 1236, exit=0, ~58s) ← 根因
│   ├── /opt/conda/bin/python (PID 1237, exit=0, ~55s) ← 镜像层 I/O 阻塞
│   ├── nvm.sh (sourced)
│   ├── nvm use default (PID 1238, exit=0, ~12s)
│   │   ├── nvm_ls v18.20.0 (PID 1239, exit=0, ~3.5s) ← 目录遍历 I/O
│   │   ├── pyenv init - (PID 1240, exit=0, ~9s)
│   │   └── pyenv rehash (PID 1241, exit=0, ~7s) ← shims 扫描 I/O
```


Stdio 捕获与 MCP 协议解析

命令行 Stdio 采集

kprobe 挂载 `vfs_write` / `vfs_writev`, 过滤 `fd=1` (stdout) / `fd=2` (stderr)

Agent 调用: Agent 任务: “迁移前备份数据库”

Agent 汇报: “Database backed up to
s3://backups/db.sql.gz”

我们采集到的 `stderr`:

Step 1: `pg_dump production | gzip > /tmp/db.sql.gz`
`stderr`: FATAL: password authentication failed
`exit code`: 2 ← dump 失败, gzip 输出 0 字节

Step 2: `aws s3 cp /tmp/db.sql.gz s3://backups/`
`stdout`: upload: ...db.sql.gz → s3://backups/db.sql.gz
`exit code`: 0 ← Agent 只看到这个

实际: 备份文件 0 字节, 迁移后无法回滚

MCP Stdio 协议解析

MCP stdio 传输模式: Agent 和 Tool (子进程) 通过 `stdin/stdout` 管道通信, 传输 JSON-RPC 2.0 消息。

```
kprobe vfs_write 捕获
→ MCPParser 解析
→ 提取:
  method=tools/call
  tool_name=read_file
  arguments={path: "..."}

```

采集链路:

① eBPF kprobe 捕获 `stdout` pipe 写入

② 内核态 `detect_jsonrpc()` 快速识别

③ 用户态 MCPParser 解析

`method/tool_name/arguments/result`

TLS 解密 — Agent 到 LLM 链路怎么打通

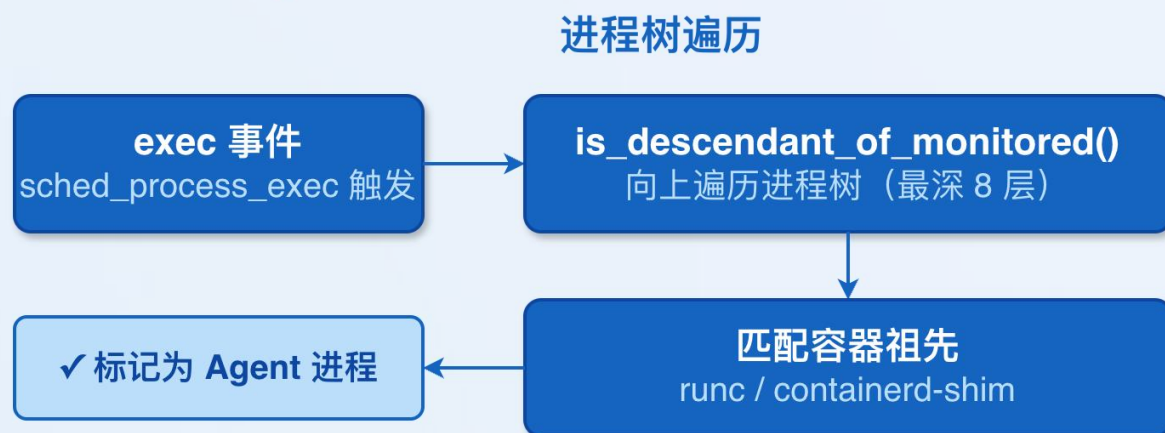
Agent → LLM 全走 HTTPS。不解密，就看不到 Prompt、Token、模型名。



加密库	状态	备注
OpenSSL/BoringSSL	支持	覆盖 Python/Node/curl 等大部分场景
Go crypto/tls	支持	goroutine 栈迁移、Register ABI、itable 变更

Agent 场景下的内核态设计

能力 1: Agent 进程发现



三模式 PID Filter (解决冷启动)



能力 2: 内核态流量分类

13 种协议推断链



流量自动分类为 5 种 Agent 通信类型



Agent 资产发现与行为风险

资产自动发现

纯被动观测，零配置声明。从网络流量中自动盘点 Agent 生态的完整资产图谱。

Agent

频繁调用 LLM 端点的工作负载，按 Namespace/Label/ServiceAccount 聚合

Tool

Agent 调用的下游服务：T-Infra / T-Data / T-SaaS

LLM Source

DNS/SNI + API 路径 + 响应特征：M-Public / M-Private

Agent 特有风险检测

Reward Hacking

Agent 报告成功，但实际未完成 → 内核证据 vs Agent 声明

权限越权

实际访问超出 RBAC 声明范围 → 行为 vs 声明偏移检测

无限重试

进程树显示相同命令循环执行 → fork 链模式识别

资源滥用

RSS/CPU 持续增长无收敛 → 进程级资源追踪

- ❏ 传统安全工具检测 syscall 异常，但缺少 Agent 语义——不知道“这个 curl 是 Agent 的正常工具调用还是数据外泄”。eBPF + Agent 意图识别解决这个问题。

后端 Agent 语义处理

能力 1 | Trace 关联



能力 2 | LLM 流量聚合



能力 3 | Session 预聚合

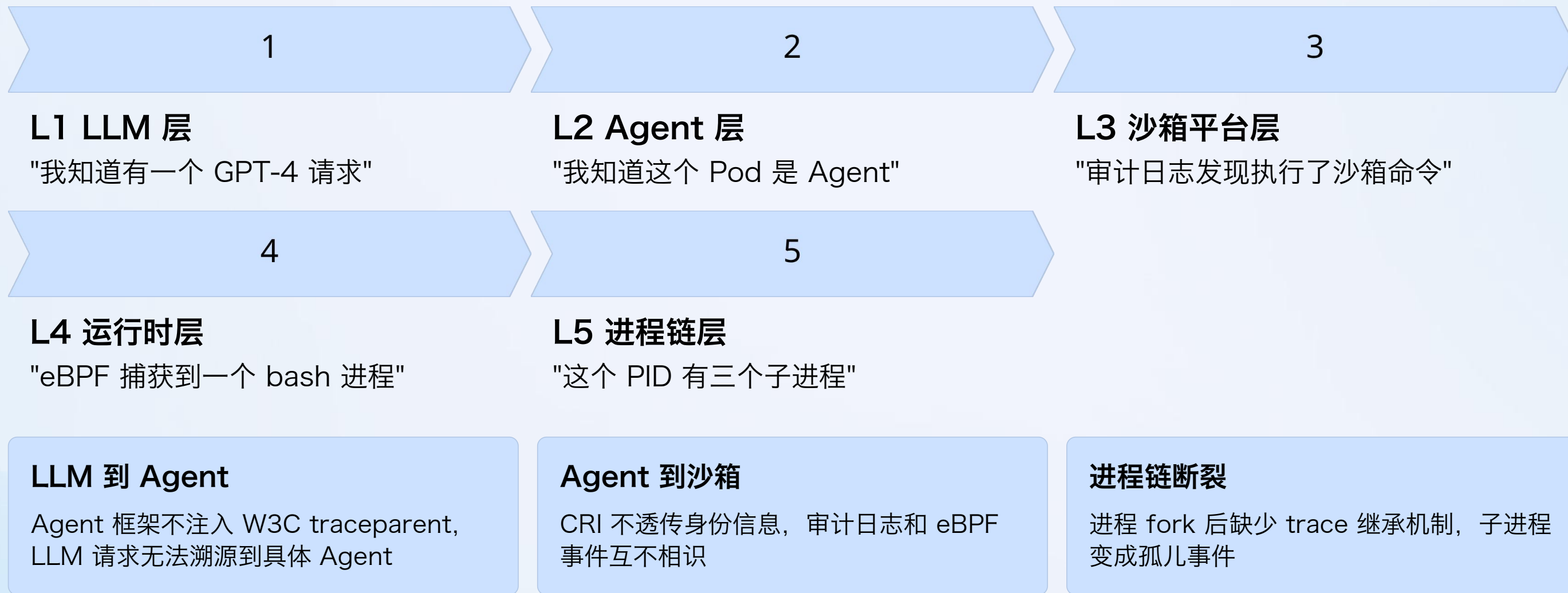


03 全链路追踪

构建从 LLM 对话到内核 Syscall 的完整视图

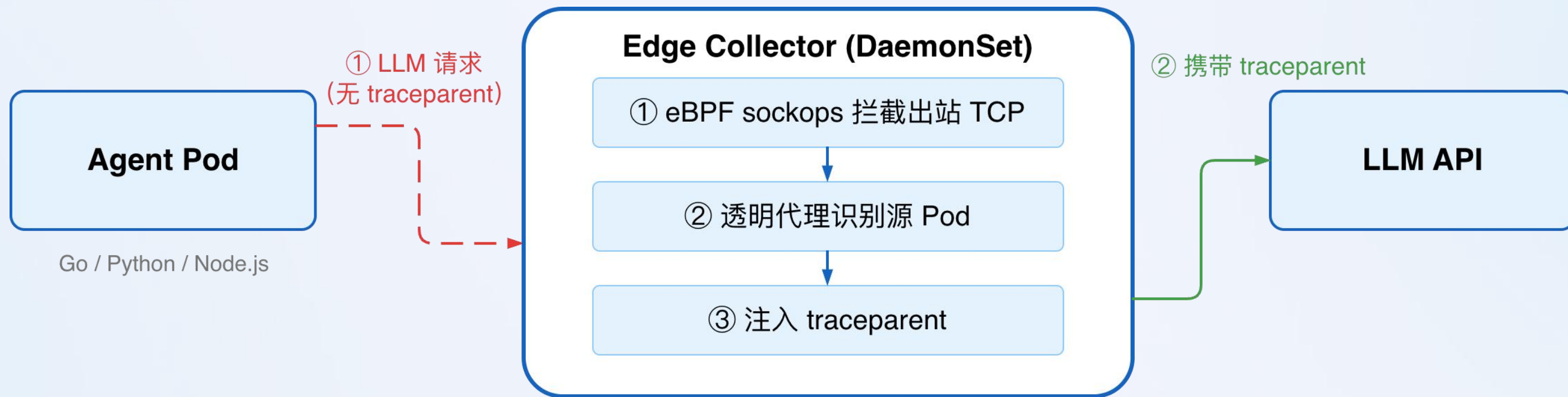
断裂问题 — 五条数据流，零条关联线

我们已经有五层采集能力：LLM 流量、Agent 资产、K8s 审计、运行时事件、进程链。但现在它们是五条独立的数据流。



LLM 层→Agent 关联 — 节点级透明 Trace 注入

问题：绝大多数 Agent 框架不注入 W3C traceparent, LLM 请求到达 API 时没有调用者标识。



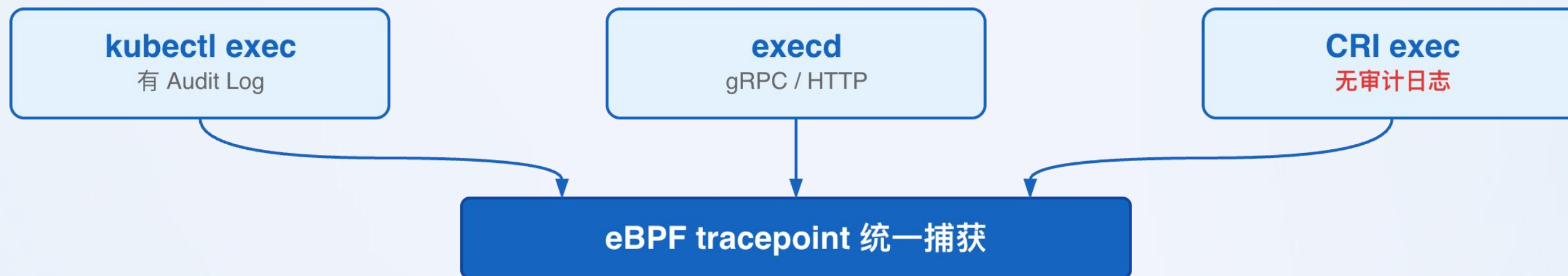
🌐 语言无关

📦 无 Sidecar

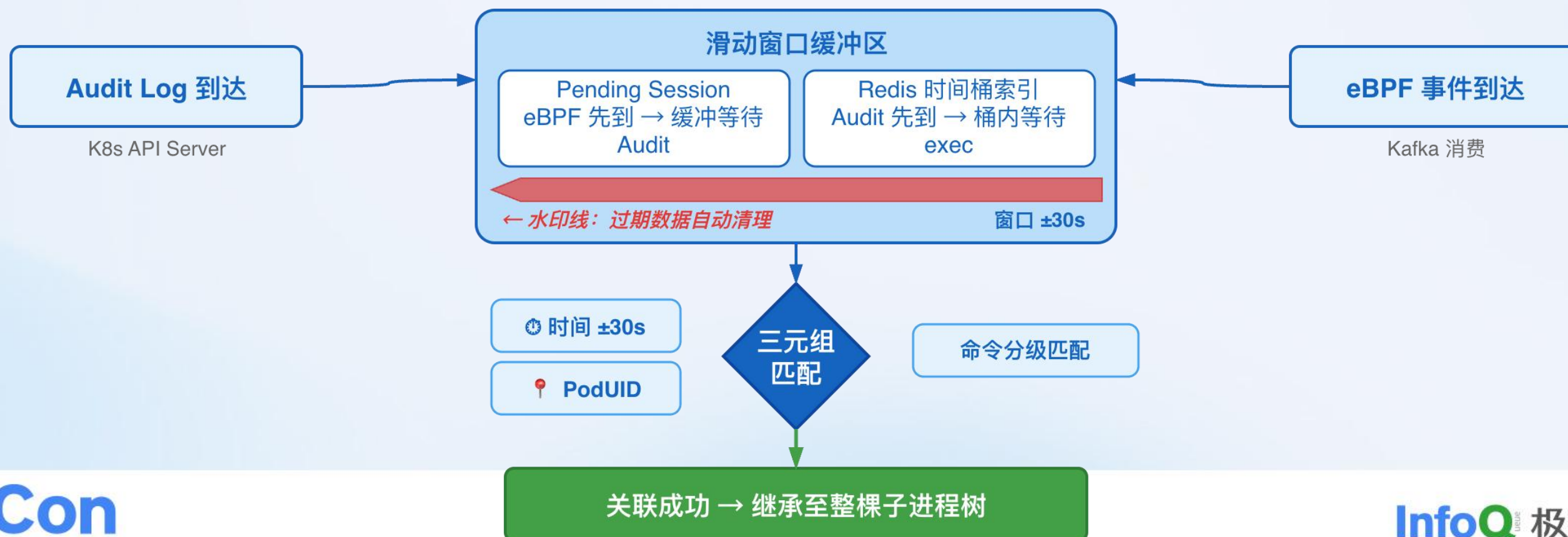
🛡️ 幂等安全

⚡ 性能影响小

沙箱平台层 → 运行时关联：沙箱入口 × 运行时事件



时空关联算法



运行时→ 进程链层关联： 进程树构建与 Trace 继承

进程树 (trace: abc-123 全树继承)



每个进程节点采集的信息

阶段	采集内容
exec (启动)	命令行 argv、可执行文件路径、启动时间
运行中	stdout/stderr、CPU time、RSS peak
exit (退出)	退出码、信号、执行时长、资源消耗汇总

一次沙箱执行触发的整棵进程树，从根到叶都在同一个 trace 下。

完整 Span 树 — Coding Agent 修复单测

trace: f7a2c1d8 · 5 层 · 13 Span · 9.4s

L1 LLM Request

Claude Sonnet · 3400 tokens · TTFT 220ms

L2 coding-agent

sandbox-prod · SA: agent-sa

L3 kubectl exec

audit_id: aud-3f7 · 审计关联 ✓

L4 bash -c "fix_and_test.sh"

PID 5012 · exit 0 · 9.4s

L5 子进程链

① cat src/utils.py

exit 0 · 0.2s

② grep -rn "def calculate" tests/

exit 0 · 0.3s

③ git diff HEAD~1 src/utils.py

exit 0 · 0.4s

④ sed -i 's/x * y/x + y/' src/utils.py

exit 0 · 0.1s

⑤ python -m pytest tests/ -v

exit 1 · 3.8s · FAILED

FAIL 3.8s

↻ Agent 读 stderr → 自我纠错

stderr → Query API → 调整策略

⑥ cat tests/test_utils.py

exit 0 · 0.1s

⑦ sed -i 's/== 15/== 8/' test_utils.py

exit 0 · 0.1s

⑧ python -m pytest tests/ -v

exit 0 · 3.6s · 5 passed ✓

PASS 3.6s

⑨ git commit -m "fix: correct calculate"

exit 0 · 0.8s

04 落地与应用

标准化输出与业务赋能

可观测指标体系 — 四大类结构化指标

原始事件经过关联和聚合后，沉淀为四大类结构化指标，覆盖 Agent 全生命周期。所有指标通过标准 Prometheus / Metrics 端点暴露，支持 ServiceMonitor 自动发现。

LLM 交互指标

- 请求量与错误率（按模型/Provider）
- Token 消耗分布（prompt/completion）
- 延迟分布（端到端 + TTFT）
- 吞吐量（RPM/TPM 滑动窗口）

执行层指标

- 事件量（exec/fork/exit 总数）
- 执行时长与退出码分布
- 沙箱执行入口检测分类
- 待完成 / 已完成 / 超时计数
- 完整的 trace 信息

eBPF 健康指标

- eBPF 程序运行次数与耗时
- CPU 占用百分比
- 已加载程序/Map 数量
- JIT 编译大小与 Map 内存

关联与处理指标

- 事件处理量与处理耗时
- 审计-执行关联成功率/失败率
- Session/Trace 去重计数
- 消息队列消费延迟与错误率

查询 API — 从 audit_id 到完整执行档案

把五层关联后的数据，通过一个统一的查询 API 暴露出来。输入一个标识符，返回完整的、跨层关联后的执行档案。

By batch_id

评测效果对比：成功/失败分布、退出码统计、资源消耗汇总

By trace_id

链路追踪：从 LLM 请求追踪到沙箱执行。返回 LLM Span + Agent 资产 + 所有 exec 事件

By session

会话回放：查看一个 Agent 会话的全部行为。返回时间排序的完整事件流

API 调用示例

```
POST /api/v1/rl/query
{
  "method": "get_trace_by_audit_id",
  "params": {
    "audit_id": "2b4a9d2b-8c1f-..."
  }
}
```

存储架构 (ClickHouse)

```
rl_sessions
→ 会话元数据 (audit_id + K8s身份)
rl_exec_records
→ 执行记录 (命令/退出码/时长)
rl_exec_stdio
→ Stdio内容 (TTL自动过期)
```

多实例架构 — 分布式状态管理

Hub 需要水平扩展，但关联引擎依赖有状态上下文。怎么在多实例间保持一致？

共享层（跨实例一致）

RedisTrace ID 缓存（TTL 24h）、
Rendezvous 会合点（TTL 2min）、
Exec 去重计数器、跨实例
Pub/Sub

本地层（实例内缓存）

进程生命周期追踪器（64 分片锁，纯
内存）、SSE 流式聚合器（按连接分
桶）、事件环形缓冲区 / TTL 去重集
合

写入层（异步批量）

Channel 缓冲 + 多 Worker /
Session 预聚合
背压策略：满载时非阻塞丢弃 + 计数
器追踪

1 共享层：Redis 优先，内存 回退

Redis 不可用时自动降级为进程内
存——功能不中断，跨实例一致性
变为 best-effort

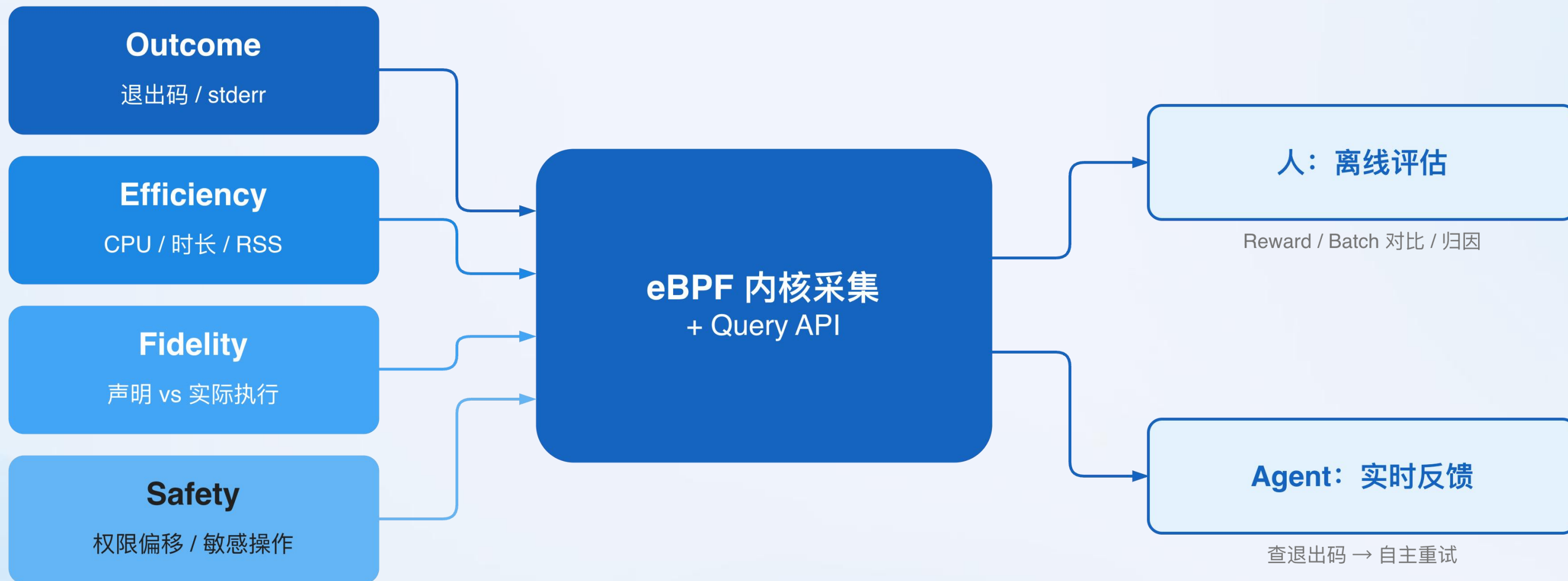
2 本地层：亲和性路由消除分 布式协调

Kafka partition key =
hash(PodUID:PID)，同一进程所有
事件落在同一 Hub 实例

3 写入层：异步批量 + 背压保 护

满载丢弃，确保采集链路不被下游
存储拖慢

观测闭环 — 从观测数据到 Agent 能力评估



Agent 性能评估与迭代 — 观测数据的工程落地



场景 1： benchmark归因

从只有结果到具体原因

将 pass/fail 拆解为进程级失败分类，区分环境、策略与资源问题，让评估结果真正可操作。



场景 2：评测间对比

版本回归定位

跨批次对比成功率与执行时长，借助 Exec Trace 定位性能劣化根因。



场景 3：Agent 自主消费

实时执行反馈

Agent 直接查询执行结果，自主判断失败原因并调整策略，形成闭环自我修正能力。



场景 4：Reward 信号

多维量化评估

将 Outcome、Efficiency、Fidelity、Safety 合成为统一 Reward，驱动 RL 训练与行为优化。

05

总结与展望



边界

eBPF 看不到 Agent 框架内部的决策逻辑

Chain-of-Thought 推理过程对内核层不可见

TLS 库版本追踪需要持续投入

OpenSSL/BoringSSL/Go 版本差异需要持续维护

VM 沙箱暂无完美的零侵入方案

Firecracker/Kata 等 VM 沙箱场景仍有覆盖盲区，需要单独部署

SSE 流式重组在极端并发下可能丢 chunk

高并发场景下流式数据完整性存在丢失可能

📌 最佳实践：分层组合 — eBPF 做底层安全网 + SDK 做上层语义增强。两者互补，而非替代。



Agent 可观测的核心问题是跨层关联，不是单层采集

APM、K8s 监控、安全工具各有专长，但跨层串联需要新方案。



Agent 生态迭代太快，无侵入方案一次建设、持续覆盖

百花齐放的 Agent 生态下的较优解。无侵入方案是唯一能让可观测能力"一次建设、持续覆盖"的架构选择



观测数据的价值不只是“人看见”

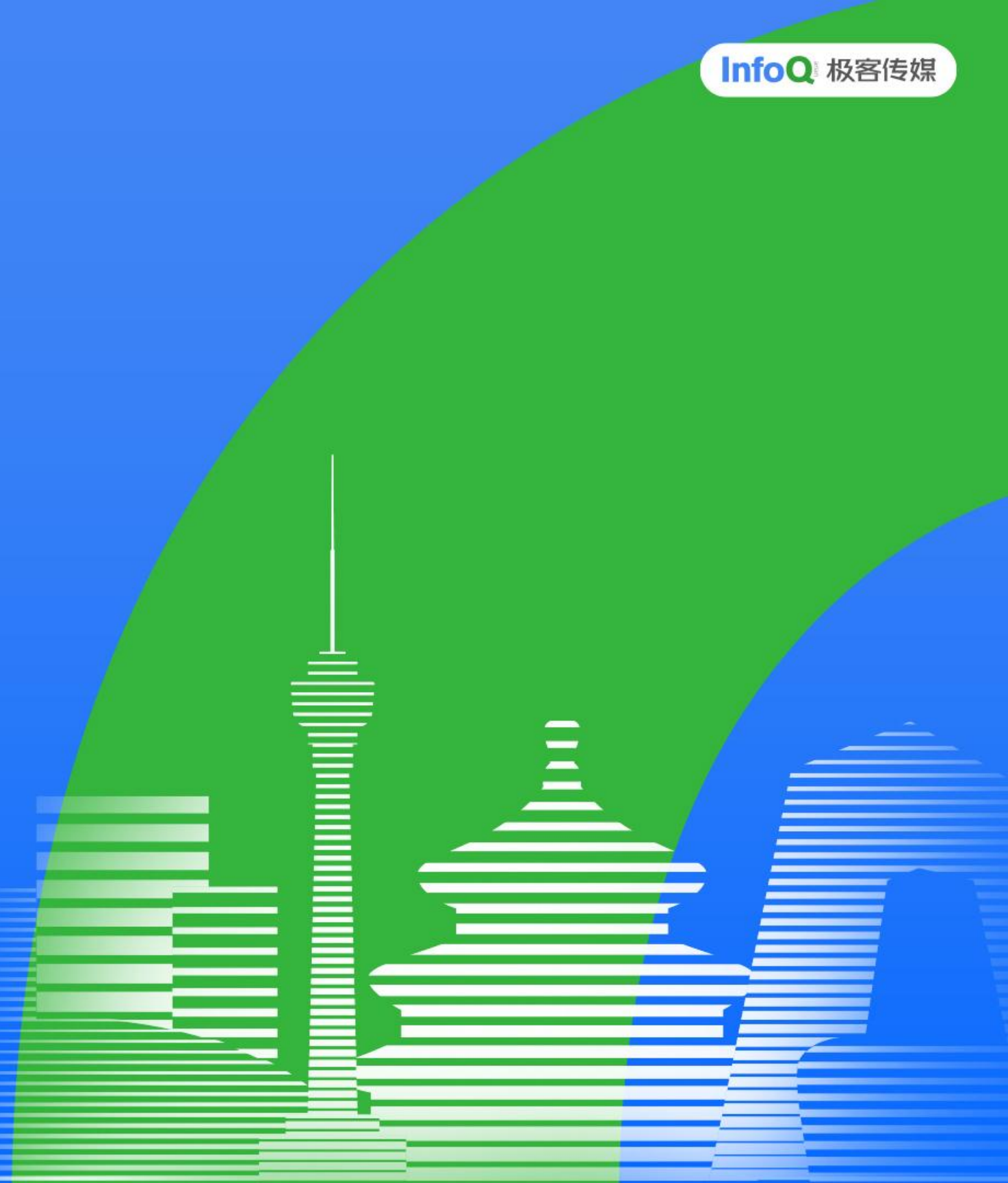
实时闭环：Agent 查退出码/stderr，自主调整策略。

长期闭环：内核级证据生成 Reward 信号，训练出更强的下一代 Agent。观测 → 量化 → 反馈 → 进化。

THANKS

大模型正在重新定义软件

Large Language Model Is Redefining The
Software



AiCon

全球人工智能开发与应用大会

上海站

探索 AI 应用边界

Explore the limits of AI applications

2026 年 6 月 26-27 日 / 上海张江科学会堂



参会咨询

专题：世界模型与多模态智能突破

专题出品人：朱政

极佳视界 / 联合创始人 & 首席科学家



专题：Agent 系统架构与工程化实践

专题出品人：鲁浩楠

OPPO / 大模型算法负责人



专题：AI 原生数据工程

专题出品人：王彦辉

火山引擎 / 数智平台产品总监



专题：Agent 安全、评测与可信治理

专题出品人：马传雷（岳立）

支付宝 / 业务风险技术部负责人



专题：Agent 数据、记忆与运行时基础设施

专题出品人：邓亚峰

EverMind / CEO



专题：企业级研发体系的重构

专题出品人：沈浪

快手 / 研发效能负责人

